Laboratório de Concorrência · Aula 10

Matheus Torrezan Izepe · RA 24170436-2 · Turma ESOFT6S-C
Gerado em 19/10/2026, 19:55:16 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando uma tarefa precisa esperar outra realizar algo para continuar. Um exemplo do dia a dia é esperar uma pessoa terminar de preparar a comida antes de começar a servir.
A sincronização de competição acontece quando duas pessoas querem usar o mesmo recurso ao mesmo tempo, então precisam esperar sua vez. Um exemplo é duas pessoas querendo usar o mesmo caixa eletrônico ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1518837

2ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1499436

3ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1593359

Saída do programa

```
esperado: 2000
obtido:   2000
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores não bateram com o esperado, que era 2000000. Os resultados foram diferentes nas três execuções, sendo 1518837, 1499436 e 1593359. Isso aconteceu porque as duas threads acessam e modificam o mesmo contador ao mesmo tempo, causando uma condição de corrida. Dependendo da ordem de execução das threads, algumas atualizações são perdidas e o resultado final muda.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

A atualização se perde quando as duas threads leem o mesmo valor antes de gravar o resultado. Por exemplo, se o contador estiver em 10, a primeira thread lê o valor 10 e soma 1, chegando a 11. Antes de gravar, a segunda thread também lê o valor 10 e soma 1, chegando a 11. A primeira thread grava 11 e depois a segunda grava 11 novamente. Assim, uma atualização é perdida, pois o resultado correto seria 12.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com n = 1000, o resultado obtido foi igual ao esperado, que era 2000. Isso aconteceu porque, nessa execução, não houve perda de atualizações perceptível. Entretanto, a condição de corrida continua existindo e pode ocorrer novamente. Esse tipo de erro é perigoso em sistemas reais porque pode não aparecer durante os testes, mas acontecer em produção, quando várias threads executam simultaneamente. Isso pode causar perda de informações e inconsistências nos dados, como valores incorretos em sistemas bancários. Por isso, é importante utilizar mecanismos de sincronização para proteger as variáveis compartilhadas.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

Saída do programa

```
print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
for valor in sorted(resultados):
    ordens = resultados[valor]
    print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")
134 ms
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens possíveis de execução e aparecem 4 resultados diferentes, que são 4, 6, 7 e 8. Apenas uma ordem resulta em 8, que é o resultado correto quando a tarefa A termina antes da tarefa B. Portanto, somente uma das 20 ordens produz esse resultado.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, o valor inicial de TOTAL é 3. A tarefa A lê o valor 3, soma 1 e grava 4. Em seguida, a tarefa B lê o valor 4 e calcula 8, mas antes de gravar esse resultado, a tarefa A já terminou. Na sequência, os passos restantes de B fazem com que ela grave um valor calculado anteriormente, resultando em 4. Assim, a operação de B é perdida porque a tarefa utiliza um valor antigo de TOTAL e acaba sobrescrevendo uma atualização com um resultado desatualizado.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Antes de executar, acredito que a versão com AtomicInteger será a mais rápida, pois realiza os incrementos de forma atômica. A versão com synchronized também deve apresentar o resultado correto, pois permite que apenas uma thread por vez execute a seção crítica. A versão com semáforo também deve funcionar corretamente se for

implementada de forma adequada. Nenhuma das três deve apresentar resultados incorretos, desde que a sincronização seja utilizada corretamente.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1955740 em 4 ms
synchronized:      2000000 em 95 ms
AtomicInteger:     2000000 em 17 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger deram o valor correto de 2000000. A versão sem sincronização apresentou 1955740, pois algumas atualizações foram perdidas devido à condição de corrida. Em relação ao tempo, a versão sem sincronização foi a mais rápida, com 4 ms, enquanto AtomicInteger levou 17 ms e synchronized levou 95 ms. As versões corretas costumam ser mais lentas porque precisam garantir que as atualizações sejam realizadas com segurança, evitando que as threads interfiram umas nas outras.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não, utilizar um AtomicInteger em cada saldo não é suficiente para garantir uma transferência bancária segura. Embora cada operação sobre um saldo seja atômica, a transferência envolve duas operações relacionadas, retirar dinheiro de uma conta e depositar na outra. Se ocorrer uma interrupção entre essas operações, os saldos podem ficar inconsistentes. Seria necessário sincronizar a transferência como um todo, garantindo que a retirada e o depósito aconteçam juntos, por meio de um mecanismo de sincronização adequado, como synchronized, além de verificar se há saldo suficiente antes de realizar a operação.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Antes de executar, acredito que as mensagens "A terminou" e "B terminou" não vão aparecer, pois ocorrerá um deadlock. A thread A vai bloquear o recurso X e esperar pelo recurso Y, enquanto a thread B vai bloquear o recurso Y e esperar pelo recurso X. Como cada thread estará esperando por um recurso que está preso pela outra, nenhuma conseguirá continuar e as duas ficarão bloqueadas.

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas

A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque cada uma segurava um recurso e esperava pelo recurso que estava sendo utilizado pela outra. A thread A tinha o recurso X e esperava pelo recurso Y, enquanto a thread B tinha o recurso Y e esperava pelo recurso X. Como nenhuma conseguia liberar o recurso que a outra precisava, as duas ficaram bloqueadas, causando um deadlock.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem elimina o impasse porque impede que uma thread segure um recurso enquanto espera por outro em uma ordem contrária. Assim, as threads conseguem acessar os recursos sem ficarem presas esperando umas pelas outras. Um exemplo de sistema real seria um sistema bancário em que duas transferências tentam acessar as mesmas contas ao mesmo tempo. Se cada transferência bloquear as contas em uma ordem diferente, pode ocorrer um deadlock. Se ambas acessarem as contas sempre na mesma ordem, esse risco é reduzido.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Antes de executar, acredito que, com 3 trabalhadores, as 6 tarefas de 100 ms serão divididas entre eles, levando aproximadamente 200 ms no total. Com 1 trabalhador, as tarefas serão realizadas uma após a outra, levando aproximadamente 600 ms. Com 6 trabalhadores, cada um poderá executar uma tarefa ao mesmo tempo, levando aproximadamente 100 ms no total. Os tempos reais podem ser um pouco maiores devido ao tempo de execução e à comunicação entre as tarefas.

Tempo com 1 trabalhador

1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

Tempo com 3 trabalhadores

3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

Tempo com 6 trabalhadores

6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

Saída do programa

```
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 4
trabalhador 1 fez a tarefa 6
trabalhador 3 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

trabalhador 5 fez a tarefa 6
trabalhador 4 fez a tarefa 5
trabalhador 2 fez a tarefa 3
trabalhador 6 fez a tarefa 1
trabalhador 1 fez a tarefa 2
```

```
trabalhador 3 fez a tarefa 4
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, o programa levou 600 ms porque as seis tarefas foram executadas uma após a outra, cada uma demorando 100 ms. Com 3 trabalhadores, levou 200 ms porque as tarefas foram divididas entre os trabalhadores, que executaram duas tarefas cada um. Com 6 trabalhadores, levou 100 ms porque todas as tarefas puderam ser executadas ao mesmo tempo. O programa não precisa de synchronized nem de trava porque as tarefas são distribuídas por meio de canais, sem precisar modificar simultaneamente uma mesma variável compartilhada.

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
        /box/main.go:9 +0x38
exit status 2

Debug this error
--
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não possui espaço para armazenar mensagens e o envio do valor 1 fica esperando que outra goroutine receba a mensagem. Como não existe outra goroutine para recebê-la, o programa entra em deadlock e apresenta a mensagem de erro. Ao mudar para make(chan int, 1), o canal passa a ter espaço para armazenar uma mensagem, permitindo que o valor 1 seja enviado sem precisar de um receptor imediato. Assim, o programa continua a execução, recebe o valor armazenado e imprime 1 na tela.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Antes de executar, acredito que o programa apresentará um erro de compilação, porque o Rust não permite que uma thread altere uma variável pertencente à função main sem garantir a segurança do acesso. Isso acontece porque o Rust verifica se há riscos de acesso simultâneo aos dados. Para permitir o compartilhamento seguro entre threads, é necessário utilizar mecanismos como Arc e Mutex.

Saídas (o erro e o resultado com move)

```
error[E0373]: closure may outlive the current function, but it borrows `contador`, which is
owned by the current function
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, o programa compila, mas imprime 0 porque a thread recebe a propriedade da variável contador, em vez de compartilhar a variável original com a função main. Assim, a thread altera sua própria versão do número,

enquanto a variável da função main permanece com o valor inicial 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Na Prática 1, em Java, o erro foi percebido durante a execução do programa, quando o contador apresentou valores menores que o esperado devido à condição de corrida. Em Rust, o compilador identifica problemas de segurança antes da execução, impedindo que determinadas operações sejam realizadas sem proteção adequada. A vantagem é reduzir erros e tornar o programa mais seguro e confiável. O custo para quem programa é precisar compreender e utilizar mecanismos como Arc e Mutex, o que pode tornar o código mais complexo e exigir mais trabalho durante o desenvolvimento.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

Antes de executar, acredito que a ordem será A, D, C e B. A e D são impressas imediatamente, enquanto C é executada pela fila de microtarefas das promessas e B pela fila de temporizadore

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem em que as letras foram impressas foi A, D, C e B. A letra B saiu depois de D e C porque o setTimeout com tempo zero não executa a função imediatamente. Primeiro, o JavaScript termina o código atual, imprimindo A e D. Depois, executa a promessa, que imprime C, e só então executa o temporizador, que imprime B.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms foi executado depois de 300 ms porque a função ocupado bloqueou a única thread durante esse período. Enquanto o cálculo estava sendo realizado, o laço de eventos não podia executar outras tarefas. Em um servidor Node.js que atende muitos usuários, um cálculo pesado pode atrasar o atendimento das requisições e aumentar o tempo de resposta. Por isso, é importante evitar bloquear a thread principal e utilizar trabalhadores para realizar cálculos demorados quando necessário.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

O problema que pode acontecer é uma condição de corrida, como na Prática 1, em Java sem sincronização. As duas pessoas podem tentar comprar o último ingresso ao mesmo tempo, fazendo com que o sistema venda o mesmo lugar duas vezes. Para evitar isso, é necessário utilizar sincronização, como synchronized, e garantir que a verificação da disponibilidade e a venda do ingresso aconteçam como uma única operação.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que duas threads podem perder atualizações mesmo realizando uma operação aparentemente simples, como incrementar um contador, e que cada linguagem oferece maneiras diferentes de evitar esse problema.